route as well, and make sure to name the path `"comment_accepted"`, as that is what the `redirect()` shortcut looks for in the `comment()` view.

Listing 7.3 A view saying Thank You when a user submits a comment

```
# RiffMates/content/views.py
...
def comment_accepted(request):
    data = {
        "content": """
            <h1> Comment Accepted </h1>

            <p> Thanks for submitting a comment to <i>RiffMates</i> </p>
        """
    }

    return render(request, "general.html", data)
```

Reuse the general.html template from the previous chapter.

There is one last step: you need the template that renders the comment form. Create RiffMates/templates/comment.html, and add the code in the following listing. This template is quite similar to the login page, using a `<form>` tag and rendering the form field using `{{form.as_p}}`.

Listing 7.4 The comment form template

```
<!-- RiffMates/templates/comment.html -->
{% extends "base.html" %}

{% block content %}
<h1>Comment</h1>

<form method="post">
  {% csrf_token %}
  {{form.as_p}}
  <button class="btn btn-dark" type="submit">Send</button>
</form>

{% endblock content %}
```

The security black magic, once again

The .as_p() method renders the fields within <p> tags.

The content template includes the `{% csrf_token %}` black magic mentioned in the previous chapter. The next section dispels the spell; for now, just make sure to include it. With all those pieces in place, you should be able to fire up your development server and visit http://localhost:8000/content/comment/. Figure 7.3 shows the resulting form.

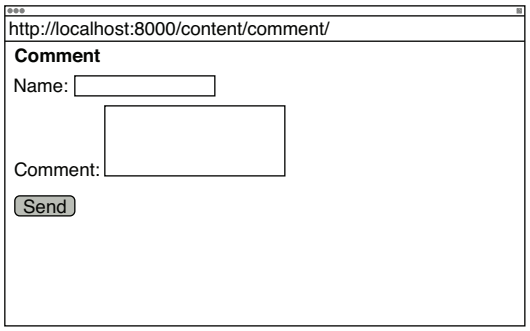


Figure 7.3 Leave a comment—we're interested, honest!